

ESPRIT AI Receptionist

Rapport de suivi de projet – Assistant vocal intelligent

Stage – ESPRIT

Document vivant, mis à jour au fil du projet
Dernière mise à jour : 12 juillet 2026

Contents

1	Introduction et contexte	2
1.1	Objectif du projet	2
1.2	Contrainte fondamentale : 100% gratuit et local	2
1.3	Répartition de l'équipe	2
2	Architecture globale	3
2.1	Les cinq modules prévus	3
2.2	Récapitulatif des modèles utilisés	3
2.3	Structure du dépôt	4
3	Infrastructure et environnement	5
3.1	Migration du disque C vers le disque D	5
3.2	Cause réelle de l'espace disque C qui se remplissait	5
3.3	Matériel et limites de performance	5
4	Module reconnaissance vocale et détection de langue	6
4.1	Choix techniques	6
4.2	Amélioration : biais de vocabulaire	6
4.3	Protocole de test qualité	6
5	Module RAG (génération de réponses)	7
5.1	Architecture	7
5.2	Amélioration 1 : détection d'ambiguïté entre programmes	7
5.3	Amélioration 2 : qualité des données – fiches "gouvernance"	8
5.4	Amélioration 3 : extraction et regroupement du dossier Admission	8
5.5	Audit de la base de connaissances : fiches peu détaillées ou hors-sujet	8
5.6	Nettoyage : préfixe d'interface web et doublons de contenu	9
5.7	Clôture de l'audit : les 4 chantiers restants traités	9
6	Module synthèse vocale (TTS)	11
6.1	Choix techniques	11
6.2	Réglage de la vitesse	11
7	Backend / API REST	12
7.1	Endpoints disponibles	12
7.2	Ce qui reste à faire côté backend	12
8	Tests et validation	13
8.1	Tests manuels effectués	13
8.2	Scripts de test disponibles	13

9	Limites connues et travail restant	14
9.1	Limites techniques actuelles	14
9.2	Travail restant	14

1. Introduction et contexte

1.1 Objectif du projet

Le projet **ESPRIT AI Receptionist** vise à concevoir un assistant vocal intelligent capable de répondre automatiquement aux appels téléphoniques des étudiants et parents de l'école ESPRIT, en français et en arabe tunisien (dialecte). L'assistant doit pouvoir comprendre une question posée à l'oral, y répondre à partir d'une base de connaissances officielle de l'école, et reformuler cette réponse à l'oral.

1.2 Contrainte fondamentale : 100% gratuit et local

L'ensemble des briques techniques utilisées sont **gratuites et fonctionnent entièrement en local** sur la machine de développement : aucune clé API payante, aucun service cloud facturé à l'usage. Ce choix structure toutes les décisions techniques du projet (choix des modèles, de l'infrastructure, des bibliothèques).

1.3 Répartition de l'équipe

- **Ikbel** : module de génération de réponses (RAG) – préparation de la base de connaissances (FAQ, web scraping à partir d'un mail, règlements, calendrier, admissions, paiements). Procédure de secours si l'information n'est pas disponible : contacter directement les services concernés (Admission : rez-de-chaussée bloc E ; FAQ : réseaux sociaux + réception entrée bloc E ; Calendrier : email sinon service scolarité ; Paiement : service financier, 1^{er} étage bloc E).
- **aziz** : backend / API REST.
- **Auteur de ce rapport** : reconnaissance vocale (STT), détection de langue, synthèse vocale (TTS), et backend / API REST.

À faire, non encore attribué explicitement : dashboard (interface CRUD documents + statistiques/indicateurs) et conteneurisation Docker + déploiement.

2. Architecture globale

2.1 Les cinq modules prévus



Module	Statut
Détection de langue (FR / arabe tunisien)	Fait – faster-whisper, 100% local
Reconnaissance vocale (STT)	Fait – faster-whisper (medium), testé
Compréhension de la question (LLM)	Fait – intégrée au module RAG
Génération de réponses (RAG)	Fait – testé et amélioré (voir chap. 5)
Synthèse vocale (TTS)	Fait – Piper, voix française
API REST (backend)	Fait – 5 endpoints fonctionnels
Dashboard (CRUD + stats/KPI)	Non commencé
Docker (conteneurisation + déploiement)	Non commencé

2.2 Récapitulatif des modèles utilisés

Étape	Modèle / outil	Détails
Détection de langue	faster-whisper (medium)	Même modèle que le STT (partagé en mémoire), CPU, quantification int8
Reconnaissance vocale (STT)	faster-whisper (medium)	100% local, device=cpu, compute_type=int8
Embeddings (recherche RAG)	sentence-transformers (paraphrase-multilingual-mpnet-base-v2)	Multilingue français + arabe, local
Base vectorielle (RAG)	ChromaDB	Stockage des embeddings, persisté dans vector_store/
Génération de réponse (RAG/LLM)	Ollama (qwen2.5:7b-instruct)	7 milliards de paramètres, 50%/50% CPU/GPU (VRAM limitée à 4 Go)

Étape	Modèle / outil	Détails
Synthèse vocale (TTS)	Piper (fr_FR-siwis-medium)	Français uniquement, 100% local

Tous ces modèles sont **gratuits, open-source, et tournent entièrement en local** (aucune clé API), conformément à la contrainte fondamentale du projet (voir section 1.2).

2.3 Structure du dépôt

data/	Collecte et preparation de la base de connaissances
raw_data/	Documents bruts (PDF, pages HTML scrapees)
processed/	Base de connaissances nettoyée (site_esprit_clean.json)
scripts/	Scripts de scraping, extraction PDF, nettoyage
modules/	
rag/	Generation de reponses (RAG)
language_detection/	Detection francais / arabe
speech_to_text/	Reconnaissance vocale
text_to_speech/	Synthese vocale
backend/	API REST reliant les modules
routers/	rag.py, stt.py, tts.py
dashboard/	(a venir)
docker/	(a venir)
tests/	Scripts de test manuels
voice_models/	Modeles de voix Piper (gitignore, telechargeable)
vector_store/	Base vectorielle ChromaDB (gitignore, regeneree)

3. Infrastructure et environnement

3.1 Migration du disque C vers le disque D

Le projet a été déplacé du disque C: vers le disque D: (d:\stage_ia_receptionist) par manque d'espace sur C. Les caches de modèles ont été redirigés vers D via des variables d'environnement Windows persistantes :

- `OLLAMA_MODELS = D:\ai_cache\ollama_models\models`
- `HF_HOME = D:\ai_cache\huggingface` (cache Hugging Face : embeddings + Whisper)

Piège identifié : un terminal déjà ouvert avant la création de la variable d'environnement ne la voit pas (les variables User ne sont lues qu'au démarrage d'un nouveau process). Toujours ouvrir un terminal neuf, ou refaire `$env:HF_HOME = "D:\ai_cache\huggingface"` manuellement si un téléchargement inattendu se déclenche.

3.2 Cause réelle de l'espace disque C qui se remplissait

Contrairement à l'hypothèse initiale (les modèles IA), la vraie cause identifiée est double :

1. **Docker Desktop** : les disques virtuels WSL2 (`docker_data.vhdx` $\approx$ 8,7 Go et `ext4.vhdx` $\approx$ 3,5 Go) vivent sur C par défaut et ne rétrécissent jamais automatiquement. Solution proposée (pas encore appliquée) : déplacer l'emplacement du disque Docker vers D.
2. **Fichiers système Windows** : `pagefile.sys` ($\approx$ 19 Go) et `hiberfil.sys` ($\approx$ 6,3 Go), gonflés par la pression mémoire (16 Go de RAM, forte utilisation par Whisper + Ollama + Docker simultanément). Solution proposée : déplacer le fichier d'échange vers D via les paramètres de performance Windows (nécessite droits admin + redémarrage).

3.3 Matériel et limites de performance

- GPU : NVIDIA GeForce GTX 1650 Ti, **4 Go de VRAM seulement**.
- Ollama (modèle 7B) tourne en partage **50% CPU / 50% GPU** (le modèle ne tient pas entier dans la VRAM).
- Whisper (STT) tourne **entièrement en CPU**, décision volontaire pour ne pas entrer en conflit de VRAM avec Ollama (voir `modules/speech_to_text/config.py`).
- Conséquence : `/api/converse` (cycle complet voix→voix) prend environ **20 à 40 secondes** par question, une fois les modèles chargés en mémoire (le tout premier appel après un redémarrage du serveur est plus lent : chargement des modèles en RAM).

4. Module reconnaissance vocale et détection de langue

4.1 Choix techniques

Basé sur `faster-whisper`, modèle `medium`, 100% local et gratuit. Les modules `speech_to_text` et `language_detection` partagent le même modèle chargé en mémoire (`modules/speech_to_text/model.py`) pour éviter de le charger deux fois.

4.2 Amélioration : biais de vocabulaire

Un paramètre `INITIAL_PROMPT` a été ajouté (`modules/speech_to_text/config.py`) pour biaiser le décodage de Whisper vers le vocabulaire du domaine (noms propres ESPRIT, termes admissions/scolarité) et quelques tournures tunisiennes courantes (باش، فم، نجم، باش، Coût nul, réversible, aucune garantie universelle – aide sur des mots spécifiques, pas sur la grammaire ou l'accent en général.

Limite connue et documentée par la recherche : Whisper (même en version large) est entraîné très majoritairement sur de l'arabe standard et quelques dialectes bien dotés en données (égyptien...). Le tunisien (derja) reste sous-représenté. Des modèles communautaires dédiés existent (`sadeemar/whisper-finetuned-Tunisian`, `SalahZa/Tunisian_Automatic_Speech_Recognition`) mais sans garantie de qualité supérieure vérifiée, ou avec un changement d'architecture non négligeable (`WavLM/SpeechBrain`). Un fine-tuning maison demanderait de collecter et annoter nos propres enregistrements tunisiens – hors périmètre réaliste du stage.

4.3 Protocole de test qualité

Un script dédié (`tests/test_language_quality.py`) guide l'utilisateur à travers 8 sujets représentatifs de la base de connaissances (admission, frais, paiement, calendrier, stage, documents, contact général, hors-sujet), enregistre chaque réponse en arabe tunisien, l'enchaîne dans le pipeline complet, et sauvegarde un rapport JSON (`Bureau\test_arabe_resultats.json`) pour analyse structurée.

5. Module RAG (génération de réponses)

5.1 Architecture

- **Base vectorielle** : ChromaDB, persistée dans `vector_store/` (regénérée via `python -m modules.rag.ingest`).
- **Embeddings** : sentence-transformers, modèle paraphrase-multilingual-mpnet-base-v2 (multilingue FR/AR).
- **LLM** : Ollama, modèle qwen2.5:7b-instruct (bon support multilingue parmi les modèles open-source).
- **Seuil de pertinence** : `SCORE_THRESHOLD = 0.45` – en dessous, redirection vers un message de fallback (service humain).

5.2 Amélioration 1 : détection d’ambiguïté entre programmes

Problème constaté : une question vague (“*combien ça coûte pour étudier à ESPRIT ?*”) piochait dans la première fiche la mieux notée, sans vérifier qu’elle correspondait bien au bon programme parmi plusieurs (cours du jour, cours du soir, EMBA...). Exemple observé : réponse de 20 000 DT (tarif EMBA) au lieu des 8 000 DT du programme d’ingénieur généraliste.

Solution implémentée (`modules/rag/programmes.py` + `modules/rag/pipeline.py`) :

1. Chaque fiche est associée à un *programme* (cours du jour tunisiens / internationaux, cours du soir, EMBA, alternance...) déduit de son URL source.
2. Si, parmi les meilleures fiches retrouvées (à une marge de score près), au moins deux programmes différents sont représentés **et** que la question ne précise pas lequel, le système répond directement par une demande de clarification – **sans appeler le LLM** (quasi instantané, ~0,3s au lieu de 15 à 40s).
3. Garde-fou anti-faux-positif : la détection ne se déclenche que si le *meilleur* résultat est déjà spécifique à un programme (sinon, une question sur un service général – stage, carrière... – ne serait pas concernée).
4. Garde-fou supplémentaire : si la question mentionne déjà explicitement un programme par mot-clé (ex. “cours du soir”), pas de clarification demandée.

Résultat validé : questions précises (“frais des cours du soir”) répondent normalement ; questions vraiment vagues (“comment puis-je payer ?”) déclenchent la clarification ; questions sur des services généraux (“service de stage”) ne déclenchent plus de faux positif.

5.3 Amélioration 2 : qualité des données – fiches “gouvernance”

Problème constaté : une question sur le département des stages ne mentionnait jamais le contact réel (Majdi Gharbi) alors que l’information existait dans la base.

Cause racine : `modules/rag/ingest.py` embed le texte `"{titre} : {contenu}"`. Pour les 28 fiches de la page gouvernance, le titre était le **nom de la personne** (ex. “Majdi GHARBI”), qui dilue l’embedding au lieu de l’aider – un nom propre n’apporte aucun signal utile à la recherche sémantique.

Correction appliquée (`data/processed/site_esprit_clean.json`, 28 fiches) :

- **Titre** : nom de la personne → sujet/département (ex. “Département des stages”).
- **Contenu** : reformulé en phrase naturelle (ex. “*Majdi GHARBI est la personne à contacter pour toute question sur département des stages. Email : majdi.gharbi@esprit.tn.*”).
- Base réindexée.

Résultat mesuré : score de similarité passé de absent du top 10 à 0,601 (2^e position) sur la question “qui contacter pour le département des stages ?”. La réponse finale mentionne désormais correctement le contact.

Limite honnête : la correction fonctionne pour les formulations proches du vocabulaire de la fiche. Des formulations plus détournées (“je cherche un stage, qui dois-je contacter ?”) ne remontent toujours pas cette fiche – limite inhérente à la recherche sémantique sur des textes courts.

5.4 Amélioration 3 : extraction et regroupement du dossier Admission

Objectif : les documents `.docx` de `data/raw_data/Admission/` (FAQ par programme, conseillers admission) n’étaient pas encore dans la base de connaissances. Plutôt que d’indexer une fiche par question isolée, le contenu est **regroupé par section** (titres en gras du document, ex. “Admission et Inscription”, “Modalités Financières”) : ni trop court (une seule question, mal retrouvée en recherche sémantique – voir amélioration 2), ni trop long (dilution de l’embedding sur plusieurs sujets différents).

Script : `data/scripts/extract_admission_docx.py` – analyseur souple gérant plusieurs variantes de mise en forme (question/réponse en paragraphes séparés ou combinés, sections purement descriptives), avec nettoyage des notes d’auteur parasites (ex. “(mettre 3 boutons cliquables...)”). Chaque fiche est rattachée au bon programme (réutilisation des URLs canoniques déjà utilisées par le système de détection d’ambiguïté, section 5).

Résultat : 27 fiches ajoutées (785 fiches au total avant l’audit ci-dessous), comparées à `site_esprit_clean.json` existant (aucun doublon exact). Test sur une question vague (“comment ça marche l’inscription aux cours du soir ?”) : réponse complète en 5 étapes – bonne, mais pas systématiquement grâce aux nouvelles fiches précises (beaucoup de contenu existant fait concurrence en recherche).

5.5 Audit de la base de connaissances : fiches peu détaillées ou hors-sujet

Un audit systématique (fiches de moins de 60 caractères, 86 sur 785) a révélé plusieurs catégories de problèmes :

1. **Conflit de calendriers** (~60 fiches) : trois fichiers calendrier différents (1,2,3&4A / 5A / 5DS9&5Infini3) avec des dates différentes pour des événements similaires (ex. vacances d'hiver) – risque de réponse mélangeant les années/promotions. **Pas encore corrigé.**
2. **Bruit PDF hors-sujet** : le document RDI_CATALOG_2023-2024.pdf (rapport annuel de recherche académique – équipes de recherche, publications scientifiques, conférences) s'est révélé **entièrement hors périmètre** pour un assistant de réception admissions/scolarité. **Corrigé** : 49 fiches supprimées (785 → 736 fiches), base réindexée, aucune régression constatée (une des nouvelles fiches "Modalités Financières" est même remontée en 1^{re} position sur un test déjà validé, probablement grâce à moins de contenu parasite en concurrence).
3. **Fiches contact avec nom en titre** (ex. "Ramla Ben Ouirane", "Pr. Faouzi Kamoun") : même défaut que la correction "Département des stages" ci-dessus. **Pas encore corrigé.**
4. **Lignes de grille tarifaire isolées** (ex. "550DT" / "Demi-pension (F1)...") : extraites une par une sans le contexte du tableau d'origine. Lié au sous-dossier Admission/Grilles tarifaires/ (PDF, Excel, zip), **pas encore traité.**

5.6 Nettoyage : préfixe d'interface web et doublons de contenu

Deux problèmes supplémentaires, découverts en creusant le mécanisme même de la recherche sémantique (comment le RAG choisit une fiche pour une question donnée) :

1. **Préfixe d'interface parasite** : 85 fiches sur 736 avaient un titre commençant par "*Ouvrir la visibilité du contenu* : " – le libellé du bouton "déplier" d'un accordéon FAQ, scrapé par erreur avec la vraie question. Comme `modules/rag/ingest.py` donne du poids au titre dans l'embedding, ce préfixe identique sur 85 fiches diluait leur pertinence. **Corrigé** : préfixe retiré (regex simple, aucun risque – ne touche jamais le contenu factuel).

2. **Doublons de contenu préexistants, révélés par le nettoyage** : une fois les titres nettoyés, ces fiches ont mieux remonté en recherche – révélant que **4 groupes de fiches quasi-identiques** existaient déjà entre plusieurs pages de programmes (ex. "Les diplômes délivrés par ESPRIT sont-ils reconnus ?" dupliqué mot pour mot entre les pages "cours-du-jour-tunisiens" et "cours-du-jour-internationaux"). Conséquence concrète : le système de détection d'ambiguïté (section 5) demandait à tort de préciser le programme, alors que la réponse est identique quel que soit le programme choisi. **Corrigé** : les 4 groupes fusionnés en une seule fiche chacun, retaguée avec une source générique non rattachée à un programme spécifique (736 → 730 fiches). Testé et validé : la question "le diplôme ESPRIT est-il reconnu ?" répond désormais directement, sans fausse demande de clarification.

Cet épisode illustre bien la nature itérative du nettoyage de données pour un RAG : corriger un problème (préfixe parasite) peut révéler un autre problème resté invisible jusque-là (doublons), simplement parce que les fiches concernées remontent différemment en recherche une fois nettoyées.

5.7 Clôture de l'audit : les 4 chantiers restants traités

1. **Fiches contact restantes** : 2 dernières fiches avec un nom de personne en titre ("Ramla Ben Ouirane", "Pr. Faouzi Kamoun") corrigées selon le même schéma que "Département des stages". La fiche du centre de carrière a ensuite été reformulée une seconde fois (titre sous forme de question "Qui contacter au centre de carrière ?") après un test montrant qu'elle perdait face à des FAQ génériques "qui contacter" d'autres programmes.

2. Conflit des calendriers : les 67 fiches issues des 3 fichiers calendrier (1,2,3&4A / 5A / 5DS9&Infini3) sont désormais préfixées avec leur promotion (ex. *“Pour 5ème année (5A) – le mardi 2025-12-30 : Vacances d’hiver.”*) : même si plusieurs dates concurrentes remontent en recherche, la réponse ne peut plus mélanger les promotions.

3. Grilles tarifaires (Admission/Grilles tarifaires/) : 7 PDF + 1 Excel extraits avec pdfplumber en mode *tableau* (pas texte brut, pour ne pas mélanger les colonnes). Le fichier zip du dossier s’est révélé être une copie exacte de 4 PDF déjà présents (tailles de fichier identiques) – ignoré. 10 fiches ajoutées : foyer et restauration (tunisiens/internationaux), historique des frais 2018-2025 (cours du jour et cours du soir), classe préparatoire (PREPA, nouveau programme ajouté au système de détection d’ambiguïté), et ESPRIT School of Business (Bachelor/Master). Chaque montant a été vérifié manuellement contre le tableau source avant d’être transcrit en phrase complète – aucune réécriture automatique de chiffres.

4. Champs source et type : vérifié dans le code que ni l’un ni l’autre n’est jamais envoyé au LLM ni à l’embedding (ingest.py et generator.py n’utilisent que titre + contenu). type était réellement mort code (jamais relu) – supprimé de toutes les fiches et des 6 scripts d’extraction. source est en revanche activement utilisé par la détection d’ambiguïté et les citations – conservé. Le vrai risque de confusion identifié était ailleurs : **203 fiches** avaient un titre brut de type *“nom-fichier.pdf - page N”* (ex. Reglement-scolarite-24-25-cs.pdf - page 8), qui pollue l’embedding (titre pesé dans la recherche). Un test empirique a confirmé qu’un titre nettoyé (ex. *“Règlement de la scolarité - Cours du soir 2024-2025”*) score mieux (0,667) qu’un titre brut (0,659) et qu’une absence de titre (0,654) – les 203 titres ont été remappés vers des noms lisibles pour les 18 documents concernés.

5. Robustesse de la détection d’ambiguïté : un cas limite a révélé que la règle “ne se déclenche que si le meilleur résultat a un label” était fragile – un écart de score de 0,001 entre deux fiches (bruit d’embedding) suffisait à faire basculer une question générale (“comment fonctionne le service de stage ?”) en fausse alerte. **Corrigé** : la règle regarde maintenant un consensus sur les 3 meilleurs résultats (majorité avec un label de programme) plutôt que le seul premier résultat – plus robuste au bruit, sans perdre la détection des vrais cas ambigus.

Base de connaissances finale : **740 fiches** (730 avant ce chantier, +10 fiches de grilles tarifaires). Régression complète validée sur 5 scénarios de référence (précis / vague / général / contact / calendrier) avant clôture de ce chantier.

6. Module synthèse vocale (TTS)

6.1 Choix techniques

Basé sur **Piper** (moteur de synthèse neuronale, 100% local, gratuit, aucune clé API). Voix française `fr_FR-siwis-medium` installée dans `voice_models/` (gitignored, retéléchargeable).

6.2 Réglage de la vitesse

La voix par défaut parlait trop vite pour un usage téléphonique. Un paramètre `SPEECH_LENGTH_SCALE = 1.25` (25% plus lent) a été ajouté dans `modules/text_to_speech/config.py`, appliqué via `SynthesisConfig` de Piper.

Limite connue : aucune voix arabe tunisienne (dialecte) disponible chez Piper – seulement de l'arabe standard moderne. Le module reste donc pertinent uniquement pour le français pour l'instant.

7. Backend / API REST

7.1 Endpoints disponibles

Endpoint	Description
GET /api/health	Vérifie que l'API répond.
POST /api/ask	Pose une question texte au RAG. Renvoie réponse + sources + indicateurs (used_fallback, needs_clarification).
POST /api/transcribe	Envoie un fichier audio, renvoie langue détectée + transcription.
POST /api/voice-ask	Envoie un fichier audio contenant une question orale, renvoie la réponse générée par le RAG (texte).
POST /api/speak	Envoie un texte, renvoie l'audio (.wav) synthétisé.
POST /api/converse	Cycle complet : audio (question) → transcription → RAG → audio (réponse).

7.2 Ce qui reste à faire côté backend

- Gestion d'erreurs explicite (fichier audio corrompu, texte vide, format non supporté).
- Tests automatisés (actuellement : validation manuelle via `curl` et `/docs`).
- Endpoints pour le futur dashboard (CRUD documents, statistiques).

8. Tests et validation

8.1 Tests manuels effectués

- `/api/health`, `/api/ask`, `/api/transcribe`, `/api/voice-ask`, `/api/speak`, `/api/converse` : tous testés de bout en bout avec succès.
- Enregistrements vocaux réels (français et arabe tunisien) via `tests/record_voice.py` puis upload manuel dans `/docs`.
- Cas limites du RAG testés : questions précises, questions vagues, questions sur des services généraux – comportement validé dans les trois cas après corrections.

8.2 Scripts de test disponibles

- `tests/test_rag_pipeline.py` – test interactif du RAG seul.
- `tests/test_stt_pipeline.py` – test interactif micro → transcription.
- `tests/record_voice.py` – enregistrement seul (sans transcription), pour tester ensuite via l'API.
- `tests/test_language_quality.py` – protocole structuré de test qualité arabe tunisien vs français (8 scénarios, rapport JSON sauvegardé).

9. Limites connues et travail restant

9.1 Limites techniques actuelles

- **Vitesse** : ~20 à 40 secondes par réponse vocale complète, due au GPU limité (4 Go VRAM) partagé entre Whisper (CPU) et Ollama (50% CPU / 50% GPU).
- **Qualité arabe tunisien** : limite structurelle des modèles gratuits disponibles (Whisper et Piper), documentée par la recherche académique.
- **Espace disque C** : cause identifiée (Docker + pagefile Windows) mais correctif pas encore appliqué (nécessite droits admin + redémarrage).

9.2 Travail restant

- **Dashboard** : interface CRUD documents + statistiques/indicateurs (nb appels selon critère, FAQ, satisfaction) – pas commencé.
- **Docker** : conteneurisation des services + déploiement – dossier vide.
- **Tests arabe approfondis** : lancer `test_language_quality.py` avec de vrais enregistrements et analyser les résultats.
- **Durcissement API REST** : gestion d'erreurs, tests automatisés.

Qualité des données (clôturé) : audit complet de `site_esprit_clean.json` terminé – fiches "gouvernance" et contacts restants corrigés, catalogue RDI hors-sujet supprimé, préfixe d'interface parasite retiré, doublons de contenu fusionnés, conflit des calendriers multi-promotions résolu, grilles tarifaires extraites et intégrées, titres "nom de fichier + page" nettoyés, champ type mort supprimé, détection d'ambiguïté rendue robuste au bruit de score. Base finale : 740 fiches.